Programming Fundamentals

Decision Making

Arab Academy
for Science, Technology and Maritime Transport

Lecture Agenda

- **Relational Operators**
- **Logical operators**
- **The If statements**
- **The If /else statements**
- **Nested If Statements**
- **The switch statements**

Relational Operators

- Relational operators allow you to compare numeric and char values and determine whether one is greater than, less than, equal to, or not equal to another.
- .Relational expressions are also known as Boolean expressions, which means their value can only be true or false.

Table 4-1

Relational Operators	Meaning
>	Greater than
<	Less than
>=	Greater than or equal to
<=	Less than or equal to
==	Equal to
!=	Not equal to

Relational Operators

- In C++, relational expressions represent true states with the number 1 and false states with the number 0.(As you will see later in this chapter, 1 is not the only value regarded as true.)

Program 4-1

```
// This program displays the values of true and false states.
#include <iostream>
using namespace std;

int main()
{
   bool trueValue, falseValue;
   int x = 5, y = 10;

   trueValue = x < y;
   falseValue = y == x;

   cout << "True is  " << trueValue << endl;
   cout << "False is  " << falseValue << endl;
   return 0;
}
```

Program Output

```
True is 1
False is 0
```

Logical Operators

- Logical operators connect two or more relational expressions into one or reverse the logic of an expression.

Table 4-6

Operator	Meaning	Effect
&&	AND	Connects two expressions into one. Both expressions must be true for the overall expression to be true.
||	OR	Connects two expressions into one. One or both expressions must be true for the overall expression to be true. It is only necessary for one to be true, and it does not matter which.
!	NOT	The ! operator reverses the "truth" of an expression. It makes a true expression false, and a false expression true.

Logical Operators

The && Operator

Table 4-7

Expression	Value of Expression
`true && false`	`false` (0)
`false && true`	`false` (0)
`false && false`	`false` (0)
`true && true`	`true` (1)

The || Operator

Table 4-8

Expression	Value of the Expression
`true || false`	`true` (1)
`false || true`	`true` (1)
`false || false`	`false` (0)
`true || true`	`true` (1)

The ! Operator

Table 4-9

Expression	Value of the Expression
`!true`	`false` (0)
`!false`	`true` (1)

Logical Operators

Precedence and Associativity of Logical Operators

Table 4-10

Logical Operators in Order of Precedence
`!`
`&&`
`||`

In the following expression, `y == z` is evaluated first, however, because the `&&` operator has higher precedence than `||`.

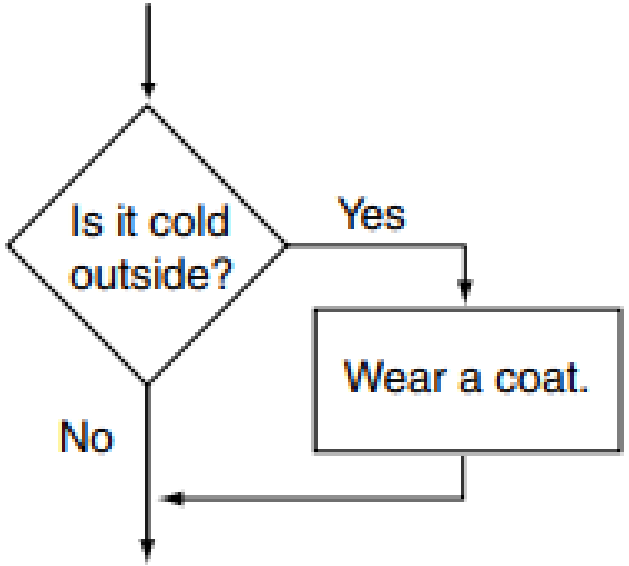
```
a < b || y == z && m > j
```

The expression is equivalent to

```
(a < b) || ((y == z) && (m > j))
```

The If statements

- The if statement can cause other statements to execute only under certain conditions



The If statements

- The general format of the if statement:

```
if (expression)
    statement;
```

- The if statement is simple in the way it works: If the value of the expression inside the parentheses is true, the very next statement is executed.

No semicolon goes here.

```
if (expression)
  statement;
```

Semicolon goes here.

The If statements

```
if (a >= 100)
    cout << "The number is out of range.\n";
```

```
if (a == b)
    cout << "Both a and b are the same.\n";
```

The If statements

- The if statement can conditionally execute a block of statements enclosed in braces

```
if (expression)
{
   statement;
   statement;
   // Place as many statements here as necessary.
}
```

```
if (average > 95)
{
   cout << "Congratulations!\n";
   cout << "That's a high score.\n";
   cout << "You deserve a pat on the back!\n";
}
```

The If statements

- What if the braces were removed???

Only this statement is conditionally executed.

```
if (average > 95)
  cout << "Congratulations!\n";
  cout << "That's a high score.\n";
  cout << "You deserve a pat on the back!\n";
```

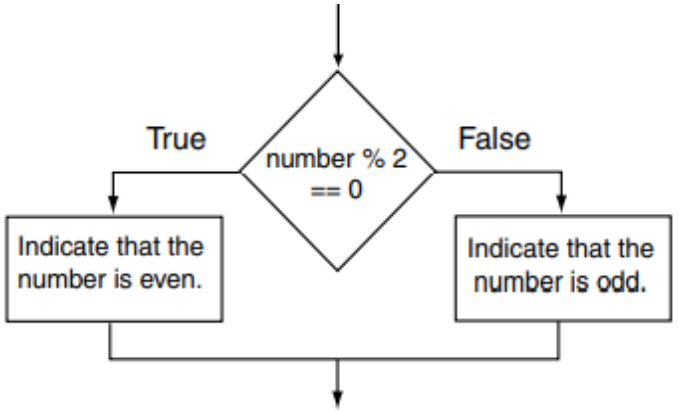
These statements are *always* executed.

The If/else statements

- The if/else statement will execute one group of statements if the expression is true, or another group of statements if the expression is false

```
if (expression)
    statement or block
else
    statement or block
```

The If/else statements



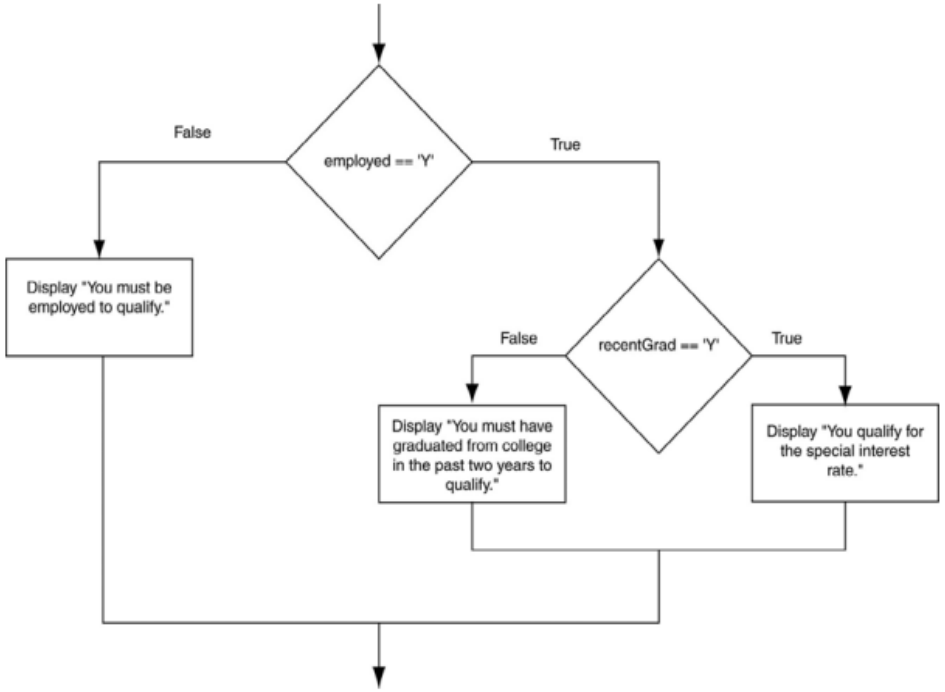
Program 4-8

```
// This program uses the modulus operator to determine
// if a number is odd or even. If the number is evenly divisible
// by 2, it is an even number. A remainder indicates it is odd.
#include <iostream>
using namespace std;

int main()
{
   int number;

   cout << "Enter an integer and I will tell you if it\n";
   cout << "is odd or even. ";
   cin >> number;
   if (number % 2 == 0)
      cout << number << " is even.\n";
   else
      cout << number << " is odd.\n";
   return 0;
}
```

Program Output with Example Input Shown in Bold

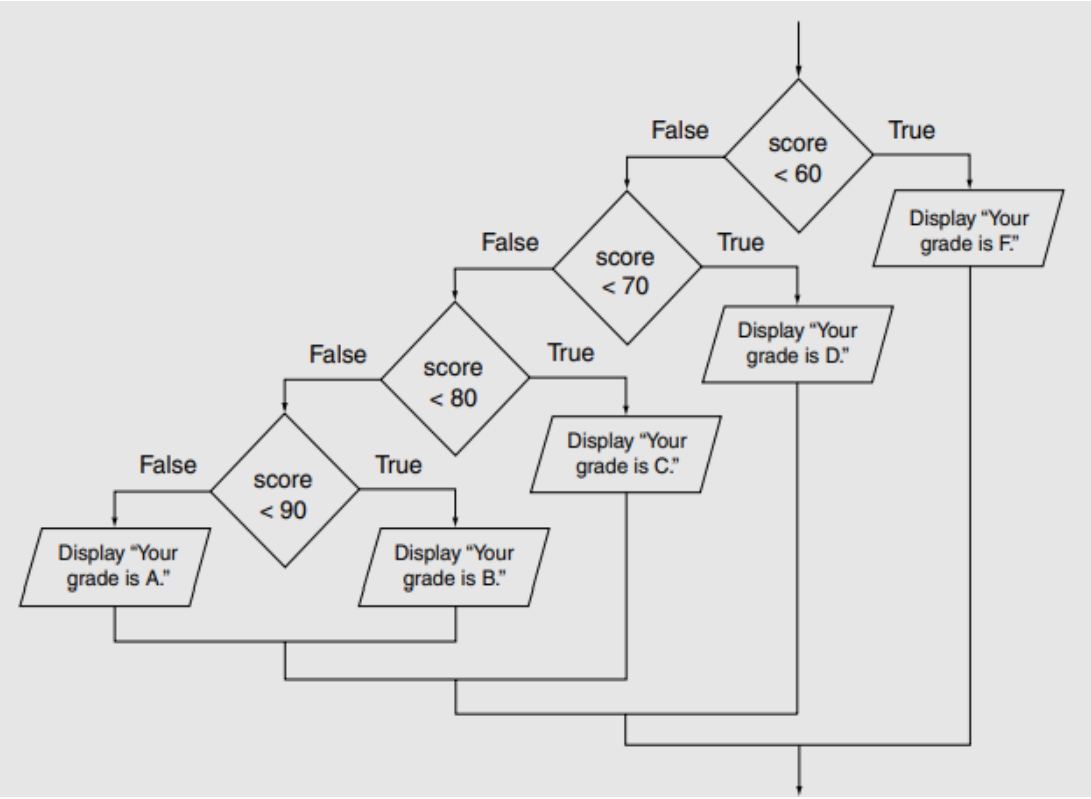
```
Enter an integer and I will tell you if it
is odd or even. 17 [Enter]
17 is odd.
```

The If/else statements




```
if (employed == 'Y')
{
   if (recentGrad == 'Y') // Nested if
   {
      cout << "You qualify for the special ";
      cout << "interest rate.\n";
   }
   else // Not a recent grad, but employed
   {
      cout << "You must have graduated from ";
      cout << "college in the past two\n";
      cout << "years to qualify.\n";
   }

  se // Not employed

   cout << "You must be employed to qualify.\n";
}
```

The If/else statements




```
// Determine the letter grade.
if (testScore < 60)
   cout << "Your grade is F.\n";
else if (testScore < 70)
   cout << "Your grade is D.\n";
else if (testScore < 80)
   cout << "Your grade is C.\n";
else if (testScore < 90)
   cout << "Your grade is B.\n";
else
   cout << "Your grade is A.\n";
```

Example (1)

Trace the following code fragments and show the output

```
int  a=5, x=10, y=25, z=30;
if(x != y ) {
  if(!(x < y) && (y < z))
        a+= x - y;
  else
        a+= x + y;
}
cout<<a;
```

Trace

a	x	y	z
~~5~~ 40	10	25	30

Output

40

Programming Challenges

The two square roots of the quadratic equation are calculated using the following equation:

$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

Write a program to check the value under the square root and the value of the denominator(2a) before calculating the roots. The program should exit if the value under the square root is negative or the value of a is zero, otherwise use the given equation to get the two roots.

Programming Challenges

```
#include<iostream>
#include<cmath>
using namespace std;
int main()
{
float  a, b, c, disc, r1, r2;
cout<<“enter the equation coefficients:\n”;
cin>>a>>b>>c;
disc = pow(b,2) – 4 * a * c;
if(a==0) cout<<“error: division by zero”;
else
if(disc<0) cout<<“imaginary roots”;
else
  {
    r1 = (-b - sqrt(disc)) / (2 * a);
    r2 = (-b +sqrt(disc)) / (2 * a);
    cout<<“root 1=”<<r1<<endl;
    cout<<“root 2=”<<r2;
  }
return 0;
}
```

The switch statement

- The switch statement lets the value of a variable or expression determine where the program will branch.
- The format of the switch statement

```
switch (IntegerExpression)
{
   case ConstantExpression:
        // place one or more
        // statements here

   case ConstantExpression:
        // place one or more
        // statements here

   // case statements may be repeated as many
   // times as necessary

   default:
        // place one or more
        // statements here
}
```

The switch statement

```
if (op=='+') result=x+y;
else
if (op=='-') result=x-y;
else
if (op=='*') result=x*y;
else
if (op=='/') result=x/y;
```

```
switch(op)
{
    case '+': result=x+y; break;
    case '-': result=x-y; break;
    case '*': result=x*y; break;
    case '/': result=x/y; break;
}
```

The switch statement

```
switch (color)
{
 case 'R':  cout<<"red\n"; break;
 case 'B':  cout<<"blue\n"; break;
 case 'Y':  cout<<"yellow\n"; break;
 default: cout<<"white\n";
 }
```

What is the output when color='B' ?

Answer:

blue

The switch statement

```
switch (color)
{
 case 'R':  cout<<"red\n"; break;
 case 'B':  cout<<"blue\n";
 case 'Y':  cout<<"yellow\n"; break;
 default: cout<<"white\n";
 }
```

What is the output when color='B' ?

Answer:

blue
yellow

The switch statement

WARNING! The expression of each `case` statement in the block must be unique.

NOTE: The expression following the word `case` must be an integer literal or constant. It cannot be a variable, and it cannot be an expression such as `x < 22` or `n == 50`.

1972 - 2022
Years Of Excellence

The switch statement

Program 4-28

```
// The switch statement in this program tells the user something
// he or she already knows: the data just entered!
#include <iostream>
using namespace std;

int main()
{
   char choice;

   cout << "Enter A, B, or C: ";
   cin >> choice;
   switch (choice)
   {
      case 'A': cout << "You entered A.\n";
                break;
      case 'B': cout << "You entered B.\n";
                break;
      case 'C': cout << "You entered C.\n";
                break;
      default:  cout << "You did not enter A, B, or C!\n";
   }
   return 0;
}
```

Program Output with Example Input Shown in Bold

```
Enter A, B, or C: B [Enter]
You entered B.
```

Program Output with Example Input Shown in Bold

```
Enter A, B, or C: F [Enter]
You did not enter A, B, or C!
```

The switch statement

Program 4-29

```
// The switch statement in this program tells the user something
// he or she already knows: the data just entered!
#include <iostream>
using namespace std;

int main()
{
   char choice;

   cout << "Enter A, B, or C: ";
   cin >> choice;
   // The following switch is
   // missing its break statements!
   switch (choice)
   {
      case 'A': cout << "You entered A.\n";
      case 'B': cout << "You entered B.\n";
      case 'C': cout << "You entered C.\n";
      default:  cout << "You did not enter A, B, or C!\n";
   }
   return 0;
}
```

Program Output with Example Input Shown in Bold

```
Enter A, B, or C: A [Enter]
You entered A.
You entered B.
You entered C.
You did not enter A, B, or C!
```

Example (2)

Trace the following program fragments and show the output:

```
int c=1, x=5, y=6, z;
switch(c)
{
    case 1: z = x % y;
    case 2: z += y / x;
    case 3: z *= x % y; break;
    case 4: z /= x % y; break;
    default: cout<<"out of range";
}
cout<<z;
```

Trace	c	x	y	z
	1	5	6	-
				~~5~~
				~~6~~
				30

Output

30

Example (3)

Rewrite the following “if statement” into “switch statement”:

```
int m, n;
if(m==1) cout<< "level one";
else
if(m==2||m== 3)  cout<<"level two";
else
if(m>=4 && m<7) cout<<"final level";
else cout<<"error";
```

Example (3)

The Answer:

```
int n, m;
switch(m)
{
case 1: cout<<“level one”; break;
case 2:
case 3: cout<<“level two”; break;
case 4:
case 5:
case 6: cout<<“final level”; break;
default: cout<<“error”;
}
```

Example (4)

Rewrite the following “switch statement” into “if statement”:

```
switch(x)
{
case 0:
case 1: cout<<“Low”; break;
case 2: cout<<“Medium”;
case 3: cout<<“High”; break;
case 4: cout<<“Extremely High”; break;
default: cout<<“Out of range”;
}
```

Example (4)

The Answer:

```
if(x==0||x==1) cout<<"Low";
else if(x==2){
        cout<<"Medium";
        cout<<"High";
}
else if(x==3) cout<<"High";
else if(x==4) cout<<"Extremely High";
else  cout<<"Out of range";
```

Programming Challenges

Serendipity Booksellers has a book club that awards points to its customers based on the number of books purchased each month. The points are awarded as follows:

- If a customer purchases 0 books, he or she earns 0 points.
- If a customer purchases 1 book, he or she earns 5 points.
- If a customer purchases 2 books, he or she earns 15 points.
- If a customer purchases 3 books, he or she earns 30 points.
- If a customer purchases 4 or more books, he or she earns 60 points.

Write a program that asks the user to enter the number of books that he or she has purchased this month and then displays the number of points awarded.

Thank You